Task 1. Gravitational Lens Substructure Classification via Modified ResNet-18

March 31, 2026

Author: Ortega Rivera Leonardo Fabyan

Abstract. This work presents a baseline approach to the classification of dark matter substructure in gravitational lensing images using a modified ResNet-18 architecture trained from scratch on synthetically generated data. The dataset comprises 37,500 single-channel images of resolution 150×150 pixels, partitioned into three balanced classes - No Substructure, SubHalo (Sphere), and Vortex - and generated using the `lenstronomy` simulation framework. The architecture was adapted to accept single-channel input and produce three-class logit outputs, and trained using the AdamW optimizer with hyperparameters selected via Bayesian optimization through Optuna. Prior to full-scale training, a structured suite of six sanity checks was conducted to validate the data pipeline, architectural integrity, and optimization stack. The model was trained for 100 epochs, achieving a validation accuracy of 93.76% and a macro-averaged F1-score of 0.9370 at the best checkpoint. Evaluation on the held-out validation set yields a mean AUC of 0.9907 across all classes under the One-vs-Rest ROC scheme, with the Vortex class presenting the greatest discriminative challenge. These results establish a strong and reproducible baseline for subsequent experimentation with more expressive architectures.

1 Dataset

The dataset comprises 37,500 single-channel gravitational lensing images of spatial resolution 150×150 pixels, partitioned into a training split of 30,000 samples and a validation split of 7,500 samples, following an 80/20 ratio. Each image is assigned to one of three mutually exclusive classes: No Substructure, SubHalo (Sphere), and Vortex. The class distribution is perfectly balanced across both splits, with each category contributing equally to the total sample count (10,000 training and 2,500 validation samples per class), thereby eliminating class imbalance as a confounding factor in model evaluation. All images were generated synthetically using the `lenstronomy` Python package, a well-established gravitational lensing simulation framework. A summary of the dataset composition is provided in Table 1. More information about the dataset, see EDA Notebook.

Split	**Total Samples**	**% of Dataset**	**Class: No**	**Class: Sphere**	**Class: Vort**
Train	30,000	80 %	10,000	10,000	10,000
Validation	7,500	20 %	2,500	2,500	2,500

Table 1: Dataset summary.

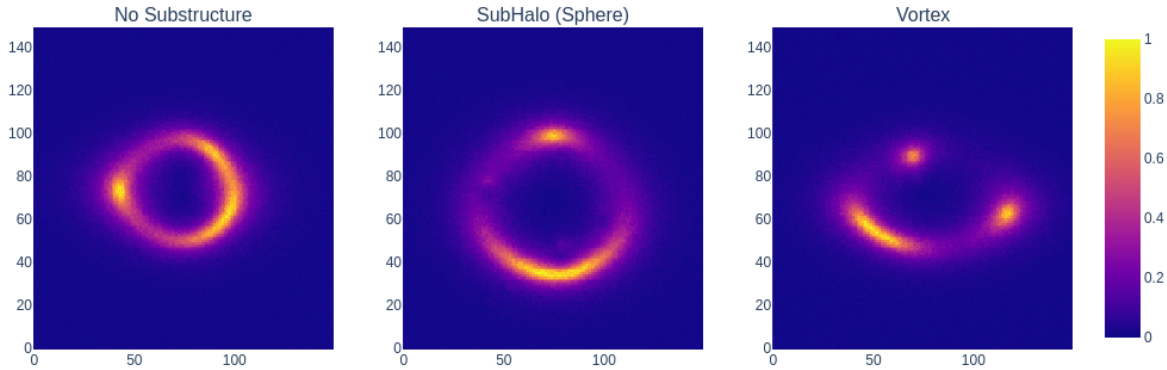

Figure 1: Example of a No Substructure, a Sphere, and a Vortex image.

2 Model

2.1 Architecture used

ResNet-18 is selected as the base architecture on account of its favorable trade-off between representational capacity and computational efficiency. Relative to deeper variants such as ResNet-50 or ResNet-152, ResNet-18 contains significantly fewer parameters, which reduces the risk of overfitting on moderately sized datasets while enabling faster experimentation and iteration. Furthermore, the residual learning framework inherently mitigates vanishing gradient issues, ensuring stable optimization even in early experimental stages. More complex architectures, such as DenseNet, introduce higher memory and computational overhead without a guaranteed performance gain for this particular task. Given the exploratory nature of this work and the controlled dataset size, ResNet-18 constitutes a robust and well-understood baseline for validating the training pipeline prior to scaling toward more expressive models. The architecture is trained from scratch, without pre-trained weights, and two targeted modifications are introduced to adapt it to the present single-channel, three-class classification problem. First, the initial convolutional layer (Conv1) is modified to accept single-channel input by reducing its in-channel dimension from 3 to 1. Second, the final fully connected layer is replaced to produce a three-dimensional output vector, mapping the learned representations to the three target classes: No Substructure, SubHalo (Sphere), and Vortex.

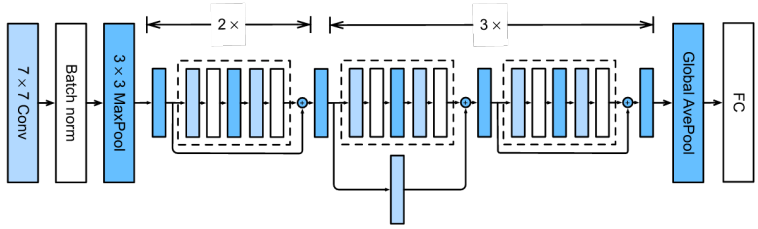

Figure 2: Original ResNet-18 Architecture. Taken from Dive into Deep Learning, Chapter 8.6: "Residual Networks (ResNet) and ResNeXt".

2.2 Sanity checks

Prior to any hyperparameter search or full-scale training, a structured suite of six diagnostic checks was conducted to verify the correctness of the data pipeline, architecture, and optimization stack. This practice, advocated by Karpathy (2019) in the context of neural network training methodology, ensures that potential implementation errors are identified and resolved before they can silently corrupt the training process or lead to misleading results. The checks are ordered by increasing proximity to the optimization loop: beginning with the data loading stage (Sections 2.2.1-2.2.2), proceeding through architectural integrity (Section 2.2.3), and culminating in optimization-level diagnostics (Sections 2.2.4-2.2.6). A failure at any of these stages would have been treated as a blocking issue requiring resolution before proceeding.

2.2.1 Batch Variance Verification

In stochastic optimization, mini-batches are assumed to constitute independent and identically distributed (i.i.d.) samples from the underlying data distribution. Violations of this assumption - such as duplicated batches, defective shuffling, or temporal correlations in the data loading pipeline - can introduce systematic bias in gradient estimates and consequently degrade convergence. To empirically validate that the DataLoader produces sufficiently distinct batches, four consecutive batches were sampled from the training set and their pairwise dissimilarity was quantified via the Mean Squared Error (MSE). The resulting similarity matrix exhibits non-zero off-diagonal entries, confirming that inter-batch variance is present and that no temporal redundancies are introduced during data loading. The diagonal entries are identically zero, as expected for self-comparisons.

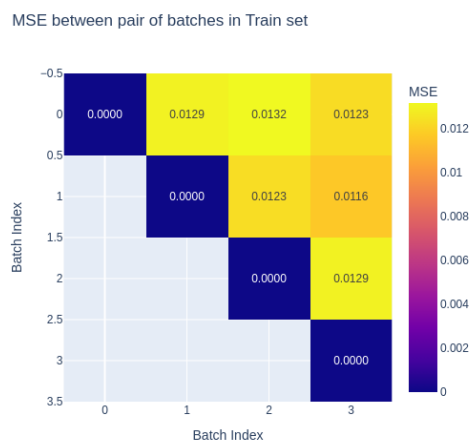

Figure 3: MSE between 4 pair of batches in Train set.

2.2.2 Augmentation Invariance Validation

Data augmentation is employed to enrich the effective training distribution and improve generalization; however, in physically grounded domains, geometric transformations may inadvertently violate the invariances of the underlying phenomenon, thereby introducing label noise. It is therefore essential to verify that the augmentation pipeline preserves class semantics prior to training. The augmentation strategy applied consists of random horizontal flips ($p = 0.5$) and random rotations uniformly sampled from the interval $[-90°, 90°]$. The assumption of rotational invariance up to 90° is physically motivated but must ultimately be validated against domain knowledge. Prior to

augmentation, all images are normalized via min-max normalization to ensure a consistent pixel intensity range across the dataset. Figure 4 presents a visual inspection of four representative samples alongside three independently drawn augmented variants per sample. The qualitative consistency between original and transformed images supports the hypothesis that the selected transformations do not alter the discriminative morphological features associated with each class.

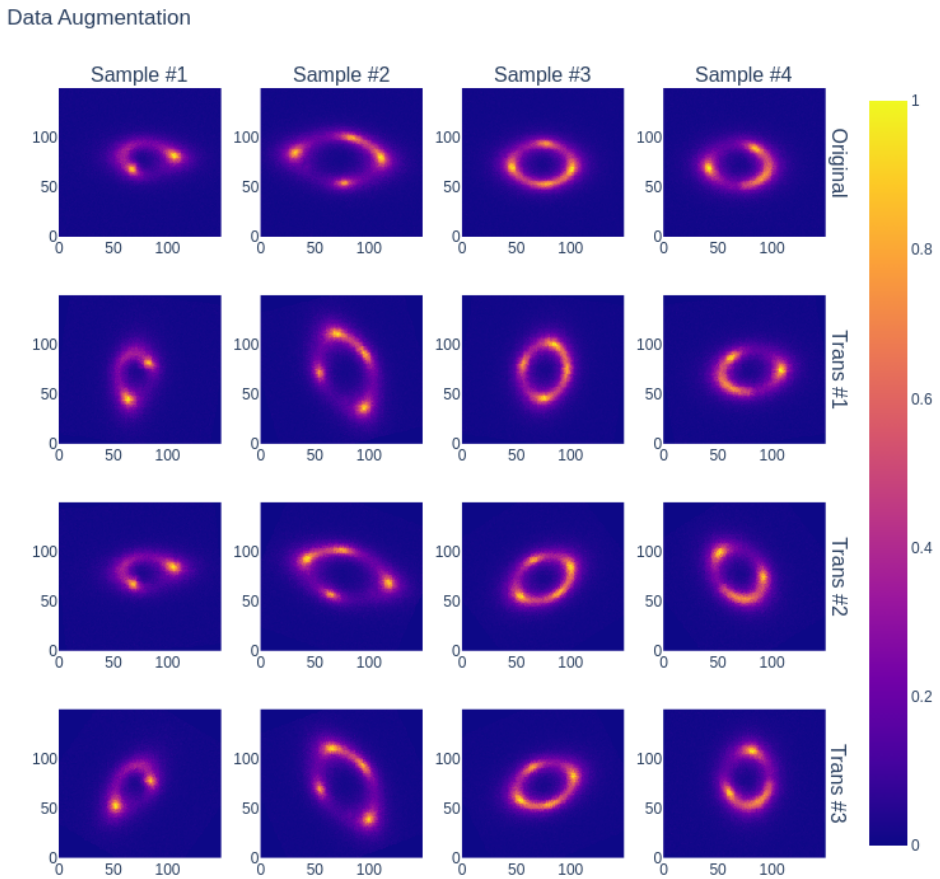

Figure 4: Example of 4 random images with 3 random Data Augmentation.

2.2.3 Forward Pass Integrity Check

Modifying a standard architecture to accommodate a different input modality and output cardinality introduces the risk of dimensional mismatches or silent broadcasting errors that may not surface until training. This step empirically verifies that the adapted ResNet-18 correctly maps input tensors to the logit space end-to-end. A single batch from the training DataLoader was passed through the model in inference mode, and the shape of the resulting output tensor was inspected. The obtained output shape of (64, 3) is consistent with the expected behavior: the batch dimension of 64 matches the configured DataLoader batch size, and the output dimension of 3 corresponds to the number of target classes. This confirms that the architectural modifications - reduction of Conv1 to a single input channel and replacement of the final fully connected layer with a three-unit output - were applied correctly, with no broadcasting errors or dimensional mismatches along the

forward pass.

2.2.4 Entropy-Based Baseline Loss Validation

Prior to any parameter updates, a correctly implemented classifier initialized with random weights should produce output logits that approximate a uniform distribution over all classes. Under this condition, the expected cross-entropy loss corresponds to the entropy of the uniform distribution over C classes

$$\mathcal{L} = -\ln\left(\frac{1}{C}\right) = \ln(C)$$

For $C = 3$, this yields a theoretical baseline of $\ln(3) \approx 1.0986$. Deviations from this value are expected to be small and attributable to random initialization variance. To empirically validate this, the cross-entropy loss was computed over the full training set using the randomly initialized model in inference mode. The observed mean loss across all batches was 1.1019, representing a relative deviation of approximately 0.003% above the theoretical baseline. This result is well within the expected range of stochastic variation and confirms that the loss function, the output layer, and the class distribution are all correctly configured before training begins.

2.2.5 Single-Batch Memorization Test

A sufficiently expressive model equipped with a correctly implemented optimization pipeline should be capable of memorizing a small, fixed dataset - that is, driving the training loss arbitrarily close to zero. This constitutes the most critical sanity check of the pipeline, as successful overfitting on a single batch provides simultaneous evidence that the architecture, the loss function, gradient backpropagation, and the optimizer are all functioning coherently and in concert.

To perform this verification, a single batch of 64 samples was held fixed and the model was trained on it exclusively for 200 epochs using the Adam optimizer with a learning rate of 1×10^{-3} and cross-entropy loss. Anomaly detection was enabled via `torch.autograd.detect_anomaly()` to surface any numerical instabilities during backpropagation. The training loss decreased monotonically and converged to 0.0 by epoch 200, confirming that the optimization pipeline is correctly implemented and that the model possesses sufficient capacity to overfit a single batch without any detected anomalies.

2.2.6 Gradient Stability and Numerical Integrity Check

Monitoring gradient statistics throughout the network is essential for detecting vanishing gradients, exploding gradients, or numerical instabilities such as NaN or Inf values, all of which can silently corrupt the optimization process. The first and last layers serve as diagnostic anchors: if the gradient of the final layer has vanished or exploded, this pathology will propagate backwards through the entire network; conversely, if the gradient of the first layer is already degenerate, it is highly likely that all intermediate layers share the same condition.

Following the single-batch overfitting experiment, the gradients of the final fully connected layer (FC) and the first convolutional layer (Conv1) were inspected for numerical integrity and magnitude. The results are summarized as follows:

- FC layer: 0 NaN, 0 Inf, mean absolute gradient $\approx 1.12 \times 10^{-5}$
- Conv1 layer: 0 NaN, 0 Inf, mean absolute gradient $\approx 5.96 \times 10^{-7}$

The absence of NaN and Inf values confirms numerical stability throughout the forward and backward passes. The observed gradient magnitudes are small but non-zero, which is expected given that the model has already converged on the single batch by epoch 200. The attenuation from the last layer to the first layer is consistent with standard backpropagation behavior through a network of this depth and does not indicate a vanishing gradient pathology in the context of this verification.

2.3 Hyperparameter optimization

The two primary optimization hyperparameters - learning rate and weight decay (L_2 regularization) - were selected via automated hyperparameter search using the `Optuna` framework with the Tree-structured Parzen Estimator (TPE) sampler. Both hyperparameters were searched over a logarithmically spaced range of $[10^{-5}, 10^{-1}]$, reflecting the several orders of magnitude over which these values can meaningfully affect convergence behavior.

Each trial consisted of training a freshly initialized model for 8 epochs on the full training set using the AdamW optimizer, with data augmentation applied inline during training. The objective metric was the mean cross-entropy loss evaluated on the validation set at the end of each trial. A total of 25 trials were conducted, with a fixed random seed of 42 applied to both the sampler and model initialization to ensure reproducibility. The optimal hyperparameter configuration identified by the search is as follows: 1.0200×10^{-4} for learning rate, 1.3699×10^{-4} for weight decay. These values were subsequently used for full-scale model training, as described in Section 3.

3 Training

The model was trained for 100 epochs on the full training set using the AdamW optimizer with the hyperparameters identified in Section 2.3: a learning rate of 1.0200×10^{-4} and a weight decay of 1.3699×10^{-4}. Data augmentation was applied inline during training as described in Section 2.2.2. All experiments were conducted on an NVIDIA GeForce RTX 2070 SUPER, with the total training duration amounting to approximately 1 hour and 16 minutes.

Training and validation metrics - cross-entropy loss, macro-averaged F1-score, and accuracy - were logged at every epoch via Weights & Biases. The model checkpoint corresponding to the lowest validation loss observed during training was saved and used for all subsequent evaluation. The best checkpoint was obtained at epoch 86, yielding the following metrics:

Split	**Loss**	**Accuracy**	**Macro F1**
Train	0.1609	0.9411 %	0.9409
Validation	0.1720	0.9376 %	0.9370

The learning curves depicted in Figure 5 show a consistent decrease in training loss alongside stable validation performance, with no evidence of severe overfitting.

4 Results

The trained model was evaluated on the held-out validation set using the best checkpoint obtained at epoch 86. Performance is assessed through three complementary diagnostic tools: the classifica-

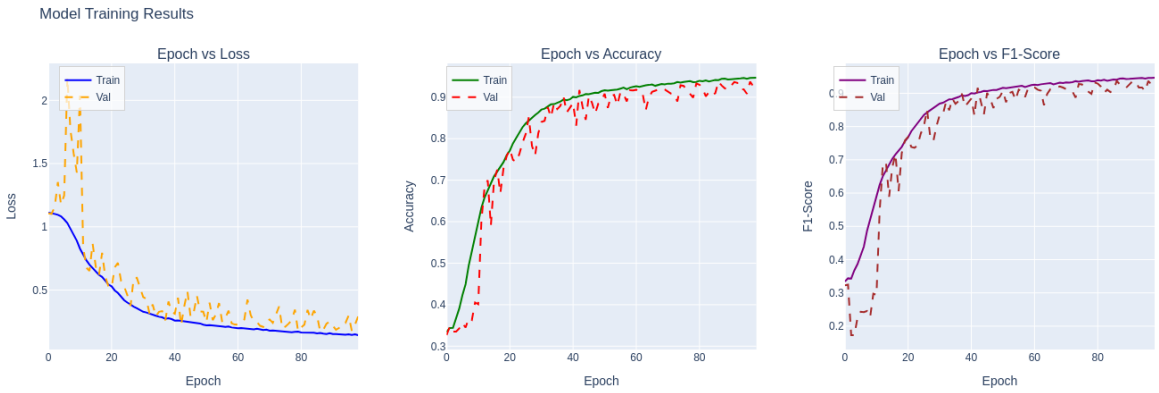

Figure 5: Epoch vs Loss, Accuracy, and Macro F1 during the training.

tion metrics reported during training (Section 3), the normalized confusion matrix (Figure 6), and the multiclass ROC curve under the One-vs-Rest (OvR) scheme (Figure 7).

The confusion matrix reveals strong classification performance across all three classes, with notably asymmetric behavior. The No Substructure class achieves a recall of 0.99, with marginal confusion distributed equally between the Sphere (0.01) and Vortex (0.00) classes. The Vortex class attains a recall of 0.96, with only minor misclassification into the No Substructure (0.02) and Sphere (0.02) classes. The SubHalo (Sphere) class exhibits the most inter-class confusion, with a recall of 0.86 and a non-negligible misclassification rate into the No Substructure class (0.09) and the Vortex class (0.05). This asymmetry suggests that the morphological features of the Sphere class present a greater discriminative challenge, likely due to its visual similarity with the other two classes under certain lensing configurations.

The ROC curves under the OvR scheme corroborate these findings. All three classes achieve high discriminative power, with AUC scores of 0.9930 (No Substructure), 0.9845 (SubHalo Sphere), and 0.9948 (Vortex). Consistent with the confusion matrix, the Sphere class yields the lowest AUC, though the margin remains small. The mean AUC across all classes is 0.9908, indicating that the model has learned highly separable representations for this three-class gravitational lensing classification task.

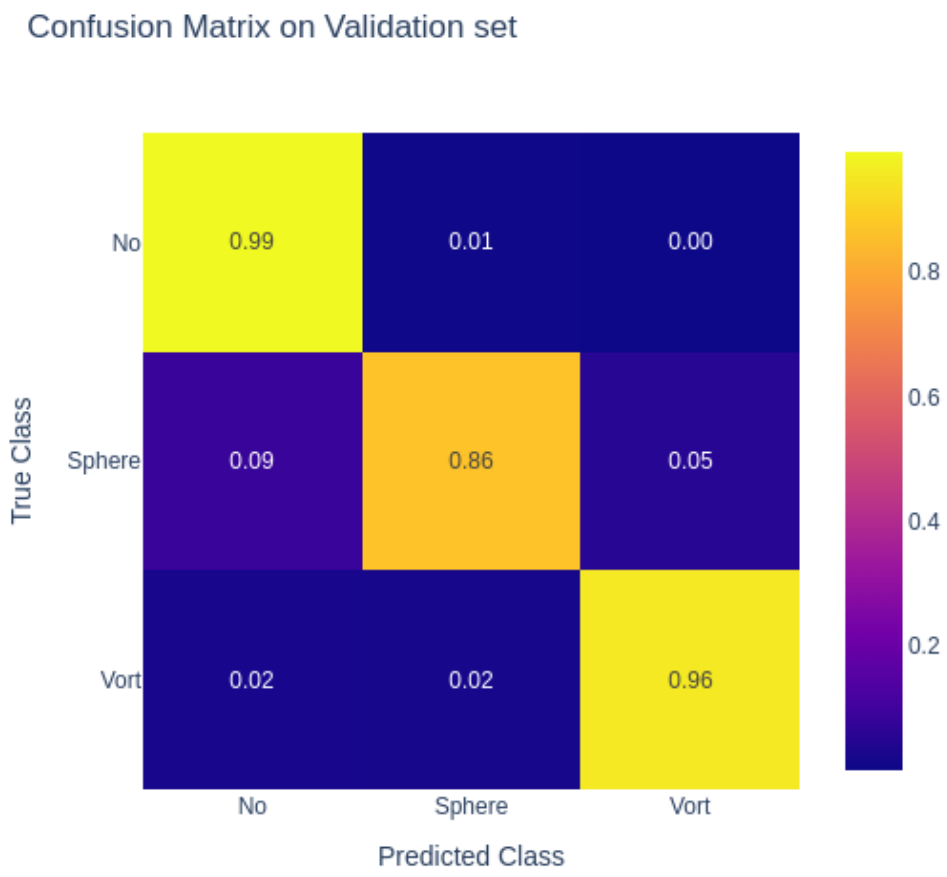

Figure 6: Confusion Matrix on Validation set.

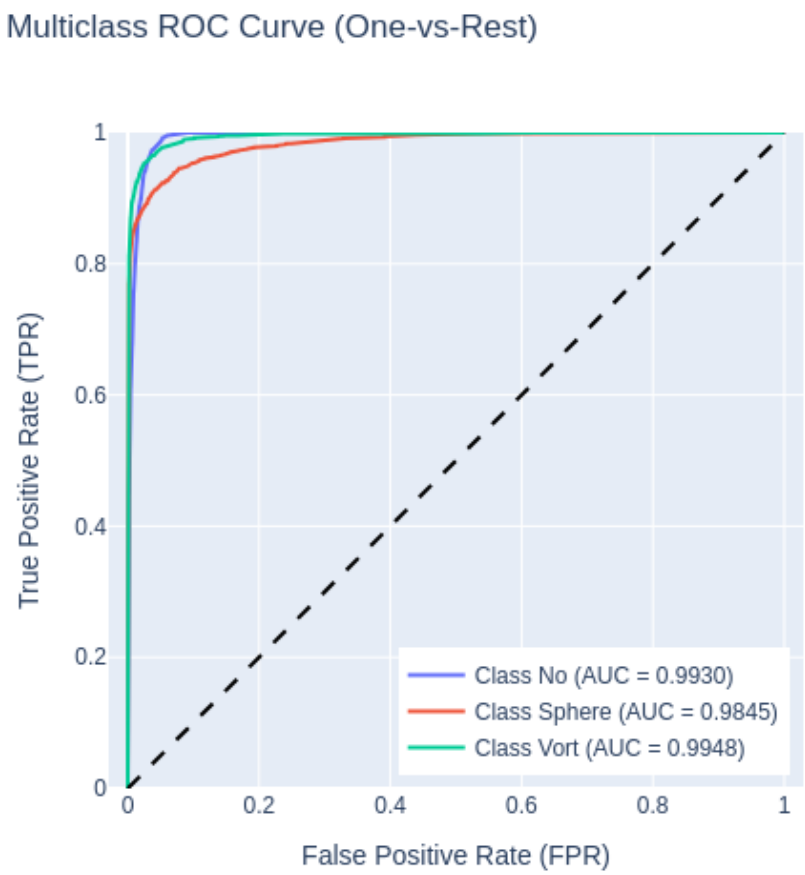

Figure 7: Multiclass ROC curve under the One-vs-Rest (OvR).